Fixed Geometric Routing from Angular Structure in Normalized Language Representations

Casey Allard
Independent Researcher

Abstract

Sparse mixture-of-experts (MoE) language models obtain computational savings by learning routers that activate only a small subset of experts per token. This paper asks a narrower question: how much of that concentration can be supplied by geometry before any router is learned? We study a fixed routing map built from a deterministic 4D chart, a Hopf-style angular parameterization, and a transport-corrected fiber phase. On a static semantic proxy, the map yields a sublinear effective routing footprint with fitted law $e_{\text{eff}} \approx 2.957 \cdot K^{0.572}$ over routing budgets $K = 25$–400, and the gap relative to a permuted fixed-routing control persists through $K = 5000$. The proxy result is sharpened by two controls: it weakens under column permutation of the embedding coordinates, and it is not reproduced by an adaptive Euclidean K-means baseline on the same structured embeddings. The same fixed route map also transfers to a two-layer language model with 64 experts: on Penn Treebank and WikiText-2 it remains within about 8% of a learned top-1 router while eliminating the learned gate matrix. The central limitation is also the central negative result: after end-to-end training, Hopf routing is not measurably better than a randomly permuted fixed routing map. The supported claim is therefore narrow. Angular structure in normalized language representations can support concentrated fixed routing, and that route map can remain usable in a small trainable model, but the present evidence does not establish a geometry-specific advantage in trained language-model quality.

1 Introduction

Modern language models rely on routing decisions at multiple levels. In a standard Transformer, each token state is mixed by attention and then processed by a dense feed-forward network (Vaswani et al., 2017). In a sparse mixture-of-experts (MoE) variant, that feed-forward block is replaced by a bank of experts and a router sends each token to one or a few of them (Shazeer et al., 2017; Lepikhin et al., 2021; Fedus et al., 2022; Lewis et al., 2021). The computational benefit comes from the fact that only a small part of the expert bank is active for any given token.

That architecture leaves open a more elementary question. Does routing concentration have to be learned from scratch, or is some of it already latent in the geometry of the representations being routed? If normalized language representations occupy angular coordinates unevenly, then a fixed angular partition could behave like a sparse router before any learned gate is introduced. Such a result would not replace modern MoE systems by itself, but it would show that part of the routing problem can be moved from optimization into geometry.

This paper studies that possibility through a deliberately narrow design. The first stage is a static semantic proxy derived from public language corpora, where the routing map itself can be evaluated without expert co-adaptation. The second stage is a small trainable language model, where the same fixed route map can be tested under end-to-end learning. The paper does not claim a full architectural replacement for learned sparse routing, and it does not claim evidence at production scale. The question is narrower: can a fixed angular address create concentrated routing on structured data, and does that route map remain usable once inserted into a trained model?

The answer is mixed. On the static proxy, the effect is strong: the effective routing footprint grows sublinearly with routing budget, persists to large K, weakens under structure-destroying permutation, and is not reproduced by an adaptive Euclidean comparator. In the trainable model, the same route map remains usable, but the geometry-specific advantage disappears: a randomly permuted fixed router performs essentially the same as the Hopf-derived one. The paper therefore makes one positive claim and one limiting claim. Fixed angular routing is a measurable effect on structured normalized representations, but the present evidence does not show that the specific Hopf-derived assignment remains uniquely beneficial after end-to-end expert adaptation.

2 Background and Scope

2.1 Sparse routing and why a fixed router is interesting

In an MoE feed-forward layer, each token state $\mathbf{h}$ is scored by a learned router, usually a small linear map followed by a softmax, and only the selected experts are evaluated. This makes routing quality an entangled object: the gate learns, the experts learn, and auxiliary regularizers may be added to keep the traffic balanced (Shazeer et al., 2017; Fedus et al., 2022). A good learned router can therefore hide several different effects at once: structure in the input states, specialization in the experts, and compensation between the two during training.

The present paper isolates the first of those effects. A fixed router is interesting precisely because it cannot adapt. If it still concentrates traffic, then the concentration must come from the occupancy pattern of the representations rather than from gate optimization. This is the same reason the negative result in the trainable model is informative: if experts can adapt to almost any fixed route map, then the geometry-specific effect is real on the proxy but not preserved as a unique quality advantage after training.

2.2 Why angular structure is plausible in language representations

Contextual language representations are not uniformly distributed on the unit sphere. Their anisotropy has been documented in trained language models (Ethayarajh, 2019), and more recent work has shown that this angular structure can have direct engineering consequences for compression (Zandieh et al., 2025). The key premise of this paper is simpler than either a compression or a large-scale systems claim. If normalized token states are angularly non-uniform, then a fixed angular map may already supply some of the traffic concentration that learned sparse routing tries to discover.

Geometry-first representations are already used to model structured data in hyperbolic embedding work (Nickel and Kiela, 2017; Ganea et al., 2018). We use a Hopf-style parameterization because it gives a compact split of a 4D chart into two kinds of information: a coarse base address and a fiber phase. The base address determines where a point lives in a low-dimensional angular manifold; the fiber phase can refine that address without changing the base location. That split is the main geometric reason this construction is worth testing. It offers a mathematically structured way to turn a normalized representation into a discrete route without learning a gate matrix.

2.3 Scope and what is not claimed

The scientific scope is intentionally narrow. The static proxy is the main geometry testbed, because it asks whether a fixed angular partition can concentrate traffic on structured language representations. The trainable language model is a transfer test: it asks whether the same fixed route map remains usable under end-to-end learning. The paper does not claim large-scale Transformer replacement, production MoE equivalence, hardware savings at deployment scale, or proof that Hopf routing is uniquely best among all fixed routers.

This boundary matters because the manuscript addresses only one empirical question. The claim retained here is limited to a fixed angular route map, a strong proxy-side concentration effect, a small-language-model transfer result, and an explicit null against a randomly permuted fixed routing map.

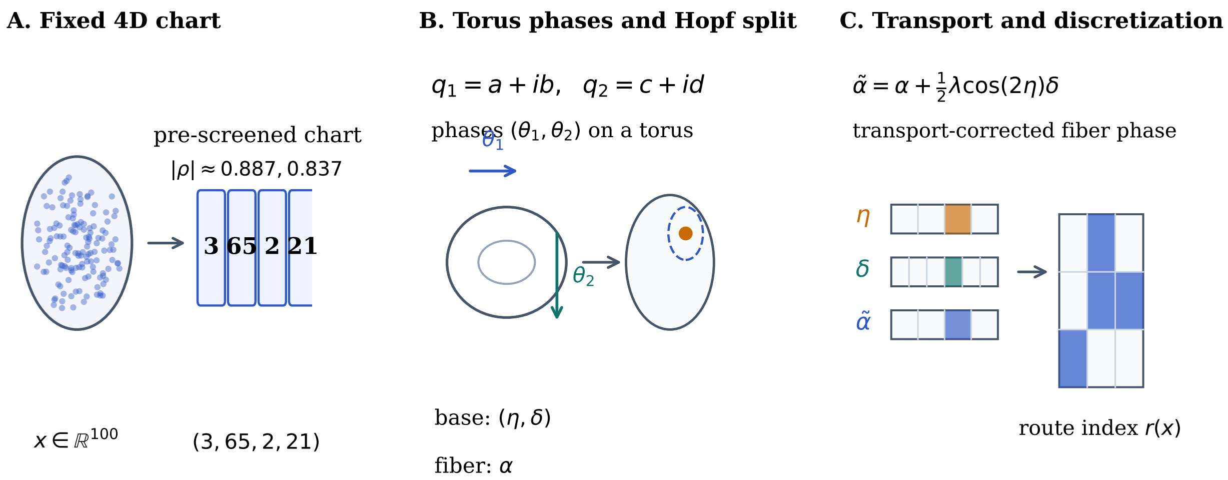

Figure 1: From normalized representation to fixed route. The router first reads a fixed 4D chart from the normalized state and forms two complex coordinates. Their phases (θ_1, θ_2) define a torus address, from which the Hopf split separates a coarse base location (η, δ) from the local fiber phase α. A transport correction produces $\tilde{\alpha}$, and the triplet $(\eta, \delta, \tilde{\alpha})$ is discretized into the final route index.

3 Method

3.1 Why a fixed 4D chart

The routing map begins with a normalized state vector $\mathbf{x} \in \mathbb{R}^D$. The paper does not learn a chart; it reads a fixed four-dimensional slice and routes from that slice alone. In the released proxy pipeline, the four dimensions are selected by a deterministic pre-screen over the structured proxy data: among the 100 PPMI-SVD features, the two most strongly correlated disjoint pairs are chosen, yielding dims $(3, 65, 2, 21)$ with absolute pairwise correlations 0.887 and 0.837. The point of this step is not to optimize a router. It is to expose a stable paired-complex chart in which angular structure can be written explicitly and then frozen across the experiments.

Figure 1 summarizes the construction. The 4D chart is not a claim about a full 4D representation of language. It is a practical device: a minimal chart in which the paired-complex geometry can be evaluated directly and identically in the static and trainable settings.

3.2 Hopf base coordinates and fiber phase

Let the selected chart coordinates be (a, b, c, d) and define

$$q_1 = a + ib, \qquad q_2 = c + id. \tag{1}$$

The two complex coordinates provide radii

$$\rho_1 = |q_1|, \qquad \rho_2 = |q_2|, \tag{2}$$

and two raw phases

$$\theta_1 = \arg(q_1), \qquad \theta_2 = \arg(q_2). \tag{3}$$

From these, the router forms three angular variables:

$$\eta = \arctan 2(\rho_2, \rho_1), \qquad \delta = \theta_1 - \theta_2, \qquad \alpha = \tfrac{1}{2}(\theta_1 + \theta_2). \tag{4}$$

For fixed η, the phase pair (θ_1, θ_2) lives on a torus. The useful split is how that torus projects under the Hopf map: δ helps determine the coarse base location, while α remains the fiber coordinate around that base point.

Each variable has a distinct role.

Table 1: Fixed angular routing procedure used in the paper.

Input: normalized state $\mathbf{x} \in \mathbb{R}^D$, fixed dims $(3, 65, 2, 21)$, budget K, transport strength λ.
1. Read $(a, b, c, d) \leftarrow \mathbf{x}_{(3,65,2,21)}$ and form $q_1 \leftarrow a + ib$, $q_2 \leftarrow c + id$.
2. Compute (η, δ, α) from Eq. (4).
3. Compute transported phase $\tilde{\alpha}$ from Eq. (6).
4. Choose balanced bin counts $(k_\eta, k_\delta, k_\alpha)$ with $k_\eta k_\delta k_\alpha \leq K$.
5. Quantize $(\eta, \delta, \tilde{\alpha})$ to integer bins $(b_\eta, b_\delta, b_\alpha)$.
6. Return route index $r(\mathbf{x}) = b_\eta(k_\delta k_\alpha) + b_\delta k_\alpha + b_\alpha$.

- η is a latitude or amplitude-split coordinate: it records how much of the 4D chart lies in the first complex plane versus the second.
- δ is a relative phase: it tells us where the point lies on the coarse angular base.
- α is a common phase: it is the fiber variable.

This is the practical consequence of the Hopf decomposition (Hatcher, 2002). The coarse base address is given by

$$h(q_1, q_2) = (\sin(2\eta)\cos\delta,\ \sin(2\eta)\sin\delta,\ \cos(2\eta))\,, \tag{5}$$

which depends on (η, δ) but not on α. In other words, the common phase disappears from the base point. This is the key geometric split behind the method. The base variables determine coarse route location, while the fiber phase can be used for internal refinement without relocating the point on the Hopf base.

3.3 Transport-corrected phase and route discretization

The canonical routing rule does not use the raw fiber phase directly. It applies

$$\tilde{\alpha} = \alpha + \frac{\lambda}{2}\cos(2\eta)\,\delta, \tag{6}$$

with fixed transport strength $\lambda = 1$. This correction is motivated by the local angular metric in Hopf-style coordinates, where the fiber and base directions are not globally independent. Operationally, Eq. (6) changes the route only when the relative-phase structure of the point makes that correction nonzero. Two points with the same average raw phase can therefore be separated if they occupy different parts of the base geometry.

The route itself is a discretization of $(\eta, \delta, \tilde{\alpha})$. Given budget K, the implementation chooses a balanced bin triple $(k_\eta, k_\delta, k_\alpha)$ with $k_\eta k_\delta k_\alpha \leq K$, then quantizes

$$\eta \mapsto b_\eta, \qquad \delta \mapsto b_\delta, \qquad \tilde{\alpha} \mapsto b_\alpha, \tag{7}$$

and returns the product-bin index

$$r(\mathbf{x}) = b_\eta\,(k_\delta k_\alpha) + b_\delta\,k_\alpha + b_\alpha. \tag{8}$$

Because the map is fixed, any concentration in route usage must come from the occupancy pattern of the inputs rather than from learned adaptation.

3.4 Effective routing footprint

The main routing metric is the *effective routing footprint*

$$e_{\text{eff}} = \exp\left(-\sum_{r=1}^{K} p_r \log p_r\right), \tag{9}$$

where p_r is the empirical fraction of tokens assigned to route r. This is the perplexity of the route-usage distribution. It equals 1 if all traffic collapses into one route and K if traffic is perfectly uniform. Lower values therefore mean more concentrated routing. For a fixed budget K, the quantity e_{eff}/K measures how much of the available routing budget is actually active.

Equation (9) is the bridge between the proxy and the trainable model. On the proxy it counts effective sectors; in the language model it counts effective expert paths. It is the same question in both cases: how many routes are genuinely in use?

4 Experimental Setup

4.1 Semantic proxy

The static proxy is built from public language data by combining two standard preprocessing steps. First, a positive pointwise mutual information (PPMI) co-occurrence matrix is computed from Penn Treebank text (Marcus et al., 1993) and reduced to 100 dimensions by singular value decomposition (SVD), following the usual matrix-factorization view of distributional embeddings (Levy and Goldberg, 2014). Second, those word embeddings are mean-pooled over length-32 WikiText-2 token windows (Merity et al., 2016) and L2-normalized, producing the proxy dataset `ppmi_proxy.npz`. The result is a structured but static representation space: there is no backpropagation, no learned gate, and no expert adaptation. This is therefore the cleanest setting in the paper for evaluating the routing map itself.

The proxy analysis sweeps routing budgets from $K = 25$ to $K = 400$ for the canonical scaling law and extends the same measurement to $K = 5000$ for a persistence check. The main outputs are e_{eff}, sector entropy, peak route mass $p_{\max}$, and comparison ratios against controls. The proxy is where the geometric question is sharpest: if the data are structured, does a fixed angular partition receive traffic non-uniformly enough to behave like a sparse router?

4.2 Trainable language-model setting

The trainable setting is intentionally small. It uses a two-layer Transformer-like language model with hidden dimension 128, four attention heads, vocabulary size 5000, sequence length 32, 64 feed-forward experts, and expert dimension 128. We evaluate on two public datasets: Penn Treebank (PTB) (Marcus et al., 1993) and WikiText-2 (WT2) (Merity et al., 2016). Each dataset is trained for 4000 optimization steps with two seeds.

Three routing conditions are central.

- Baseline: a learned top-1 router with no auxiliary load-balancing loss.
- Hopf: the learned router is removed and replaced by the fixed route in Table 1.
- Permuted: the same fixed route map is kept, but the sector labels are randomly permuted.

The Permuted condition is the main null. It preserves parameter count, route budget, and fixed sparsity, but removes any geometry-specific sector assignment. If Hopf and Permuted are equivalent after training, then the trainable evidence cannot be read as proof that the specific geometry remains uniquely useful under expert co-adaptation.

4.3 Controls and falsification logic

The interpretation of the paper depends on four controls.

- **Structured vs. column-permuted proxy input.** This destroys alignment between coordinates and semantic structure while preserving marginals. If the main proxy effect survives unchanged, the result would not be specific to structure.
- **Hopf vs. adaptive Euclidean K-means on the proxy.** This asks whether any adaptive clustering reproduces the same concentration.
- **Hopf vs. permuted fixed routing in the trainable LM.** This is the central null test for the trainable setting.
- **Auxiliary-loss control for the learned comparator.** A balance-forcing objective is included only to test whether it is the wrong comparison regime for a geometry that is supposed to concentrate traffic natively.

The controls do not all answer the same question. The first two ask what was falsified on the proxy. The third asks what failed to transfer to the trainable setting. The fourth constrains how the learned baseline should be chosen.

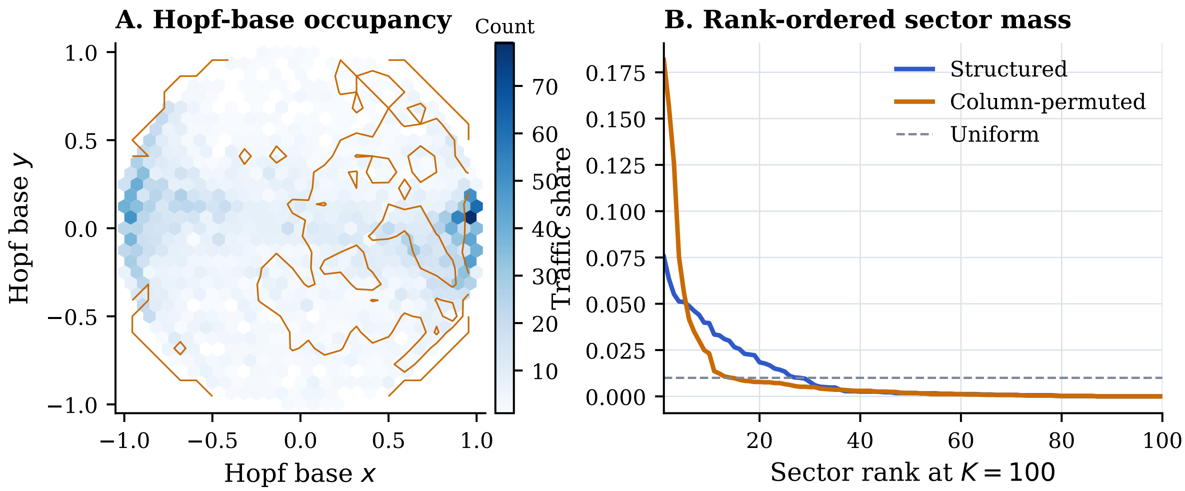

Figure 2: Why a fixed angular partition can behave like a sparse router. Left: structured states occupy the Hopf base unevenly, unlike the column-permuted control. Right: after sectorization at $K = 100$, the structured proxy places more traffic in the top-ranked sectors.

5 Semantic Proxy Results

5.1 Why the geometry can concentrate traffic

Figure 2 shows the mechanism visually before it is reduced to a single scaling exponent. The left panel projects 5000 structured proxy states onto the Hopf base. The occupancy is visibly uneven: some parts of the base receive dense traffic while others remain sparse. The right panel converts that geometry into route usage at $K = 100$. A small subset of sectors carries a large fraction of the traffic, whereas the column-permuted control spreads mass more evenly across ranked sectors.

This occupancy pattern is the mechanism by which a fixed angular partition can behave like a sparse router. A fixed partition only becomes useful if the data visit some cells much more often than others. The static proxy shows exactly that behavior. The geometry is doing no optimization; it is simply exposing an occupancy pattern that is already present in the normalized representations.

5.2 The main scaling law

Figure 3 reports the main proxy result. On the structured proxy, the effective routing footprint grows sublinearly with routing budget. Over the anchor range $K = 25$–400, the fitted law is

$$e_{\text{eff}} = 2.957K^{0.572}. \tag{10}$$

The corresponding permuted fixed-routing control grows much faster, with exponent 0.814. The same pattern persists at larger budgets: by $K = 5000$, the structured-vs-permuted gap reaches 2.64×.

The right panel of Figure 3 helps interpret the exponent. The question is not whether the structured proxy uses fewer routes than dense routing at a single budget; that is guaranteed. The real question is whether finer partitions continue to create new active routes nearly linearly. They do not. The occupancy gap widens with K, which is why the fit exponent is substantially below one.

5.3 What the proxy controls rule out

The main scaling law would be much weaker without its controls.

First, the structure-destroying proxy control weakens concentration. In the finalized semantic discrimination experiment, peak sector mass drops from 0.090 on the structured proxy to 0.061 after column permutation, and the route entropy rises accordingly. This rules out the interpretation that the sublinear footprint is merely a trivial consequence of fixed assignment.

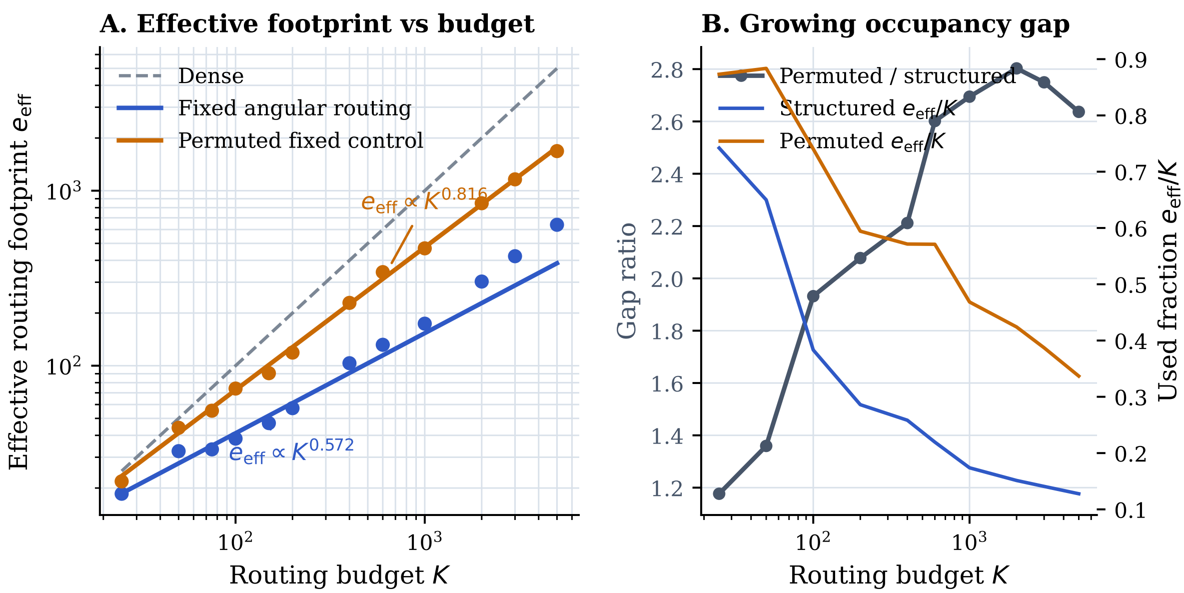

Figure 3: Main proxy result. Left: the effective routing footprint of the fixed angular map grows sublinearly, unlike dense routing and unlike the permuted fixed-routing control. Right: the gap widens as the routing budget increases because the structured proxy keeps using a shrinking fraction of the available cells.

Second, the adaptive Euclidean control does not match the geometric one. On the same structured proxy, fixed Hopf routing reaches peak sector mass 0.087, whereas adaptive K-means reaches 0.072. The appendix reports the full table. This is a key proxy-side control: a high-dimensional adaptive Euclidean baseline does not reproduce the fixed Hopf concentration effect.

Third, the scaling law is essentially norm-invariant. Across L1, L2, L3, and L4 normalization variants, the fitted Hopf exponent varies by less than 0.000. Activating radial shells does not strengthen the concentration signal. This narrows the interpretation further: on the current proxy, the effect is angular rather than radial.

Taken together, these controls justify a concrete statement about what was falsified on the proxy. The evidence is inconsistent with a generic "any fixed partition will do" story, inconsistent with a generic adaptive Euclidean clustering explanation, and inconsistent with the claim that radial shell structure is carrying the result. What remains is a specifically angular occupancy effect on structured normalized representations.

6 Results in a Trainable Language Model

6.1 Transfer of the fixed route map

Table 2 summarizes the trainable-language-model results. On PTB, the HOPF/BASELINE validation-perplexity ratio is 1.081. On WT2, it is 1.081. Across both datasets, the fixed route map remains within about 8% of the learned top-1 baseline while removing the learned gate matrix.

The routing side of the transfer result is weaker than on the proxy but still meaningful. Hopf uses 46.0 effective expert paths on PTB and 40.9 on WT2 out of 64 available experts. In other words, the fixed route map remains usable inside the model without catastrophic expert collapse, even though the quality gap relative to the learned router remains visible.

The appendix includes PTB training curves for both validation perplexity and effective expert-path usage. Those curves show the same pattern as the final metrics in Table 2: the fixed routers remain usable throughout training, but the Hopf and permuted conditions track each other closely. The fixed routing effect transfers; the geometry-specific advantage does not.

Table 2: Main trainable-model result. Fixed angular routing is usable in a two-layer language model on both PTB and WT2, but the Hopf-derived assignment does not outperform the permuted fixed-routing null.

Dataset	Router	Val. PPL	H/B	Eff. paths	Params
PTB	BASELINE	152.26	1.000	43.2	5.66M
PTB	HOPF	164.54	1.081	46.0	5.64M
PTB	PERMUTED	164.41	1.080	45.8	5.64M
WT2	BASELINE	105.85	1.000	35.4	5.66M
WT2	HOPF	114.45	1.081	40.9	5.64M
WT2	PERMUTED	114.48	1.082	42.3	5.64M

6.2 The null result against permuted routing

This null result is central rather than incidental. In PTB, the final difference between Hopf and Permuted is +0.13 perplexity points. In WT2, it is -0.03. Those gaps are too small to support a claim that the Hopf-derived sectorization remains uniquely better after end-to-end training.

Scientifically, this changes the meaning of the paper. The static proxy shows that the specific geometry matters when the router itself is the object under test. The trainable language model shows that expert adaptation can absorb that specificity. What transfers is the existence of a usable fixed sparse route map, not a geometry-specific quality advantage of the Hopf assignment after learning.

6.3 Why the learned comparator is no-aux top-1 routing

The learned comparator is a top-1 router without an auxiliary load-balancing loss. That choice is deliberate. The paper is not asking whether a fixed angular route map can mimic a balance-forced router. It is asking whether that route map can replace a naturally concentrated learned sparse regime. The auxiliary-loss control in the appendix shows why this matters: once the balance penalty is introduced, the learned router is driven toward near-uniform expert use, reaching 62.8 effective paths out of 64. That control does not invalidate the fixed-routing result; it clarifies the comparison target. For the concentration question asked here, the no-aux learned top-1 baseline is the appropriate comparator.

7 Discussion and Limitations

The paper supports one positive conclusion and one negative conclusion.

The positive conclusion is that angular structure in normalized language representations can behave like a routing resource. On the static proxy, a fixed angular partition yields a clearly sublinear routing footprint, survives strong controls, and continues to concentrate traffic at large routing budgets. In the trainable model, the same fixed route map remains usable without a learned gate matrix.

The negative conclusion is that the geometry-specific advantage does not survive end-to-end expert adaptation at this scale. The trainable evidence does not show that the Hopf-derived assignment is better than a randomly permuted fixed route map. This is not a minor caveat. It is the main reason the paper does not claim that fixed geometry has already replaced learned sparse routing in modern language models.

Several further limitations follow directly.

- The evidence is strongest on the static proxy rather than in the trainable model.
- The trainable architecture is deliberately small: two layers, 64 experts, short runs, and feed-forward routing only.
- The paper shows a routing consequence of angular structure, not a full architectural consequence for Transformers, compression, or hardware systems.
- The broader research program that motivated the work remains outside scope here.

These limitations define the claim that actually survived the controls. A fixed angular routing effect is real, measurable, and transferable to a small trainable language model, but the current evidence does not justify a stronger public claim.

8 Conclusion

This paper asked whether a fixed routing map derived from angular structure can supply some of the concentration normally provided by a learned sparse router. The answer is yes in the static setting and only partially in the trainable one. On a structured semantic proxy, the fixed angular map yields a sublinear effective routing footprint and survives controls that make its interpretation much sharper. In a small trainable language model, the same route map remains usable without a learned gate matrix, but its geometry-specific advantage disappears against a permuted fixed-routing null. The resulting empirical claim is narrow but concrete: angular structure in normalized language representations can support concentrated fixed routing, and that routing scaffold remains usable in a small trainable model, although the present evidence does not show a geometry-specific quality advantage after end-to-end training.

References

Kawin Ethayarajh. How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of EMNLP-IJCNLP*, 2019.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23:1–39, 2022.

Octavian-Eugen Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. In *Advances in Neural Information Processing Systems*, 2018.

Allen Hatcher. *Algebraic Topology*. Cambridge University Press, 2002.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Chen, Gael Fried, and Katherine Mahowald. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.

Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. In *Advances in Neural Information Processing Systems*, 2014.

Mike Lewis, Luke Zettlemoyer, Edouard Grave, Armand Joulin, and Shruti Bhosale. BASE Layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning*, 2021.

Mitchell P. Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems*, 2017.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. *arXiv preprint arXiv:2504.19874*, 2025.

A Detailed Experimental Setup

Table 3 consolidates the exact settings used for the static proxy and the small trainable language model.

Table 3: Exact settings for the experiments reported in the paper.

Item	Value
Static proxy	PPMI-SVD embeddings (100 dimensions), context length 32
Proxy source text	PTB co-occurrence statistics applied to WT2 token windows
Proxy sample counts	50,000 train / 50,000 val / 50,000 test
4D routing chart	dims $(3, 65, 2, 21)$ from proxy correlation pre-screen
LM architecture	2 layers, hidden 128, 4 heads, sequence length 32
Experts	64 experts, expert dimension 128
Datasets	PTB and WikiText-2
Training horizon	4000 steps, 2 seeds per dataset
Main LM conditions	BASELINE, HOPF, PERMUTED
Primary routing metric	$e_{\text{eff}} = \exp(-\sum_r p_r \log p_r)$

The proxy dataset used in this paper contains 50,000 examples in each split, 100-dimensional normalized inputs, and 256-dimensional hashed regression targets. The accompanying routing implementation follows the procedure in Table 1 directly and reproduces the proxy-side scaling law from the distributed data files.

B Additional Proxy Controls

Table 4: Structure-destroying proxy control. Column permutation weakens concentration even though marginal feature statistics are preserved.

Proxy condition	Runs	Sector $p_{\max}$	Sector entropy	Test MSE
Structured	4	0.090	3.005	0.0040
Column-permuted	4	0.061	3.187	0.0040

Table 5: Adaptive Euclidean control. Fixed Hopf routing is more concentrated than adaptive K-means on the same structured proxy.

Routing family	Runs	Sector $p_{\max}$	Sector entropy	Test MSE
Hopf	2	0.087	3.005	0.0040
K-means	2	0.072	3.058	0.0040
Hopf permuted	2	0.064	3.183	0.0040
K-means permuted	2	0.070	3.182	0.0040

Table 6: Normalization robustness. The fitted Hopf exponent changes only minimally across L1, L2, L3, and L4 normalization variants, which supports the angular interpretation of the proxy result.

Normalization	Hopf exponent	Permuted exponent	Base exponent
L1	0.572	0.809	0.916
L2	0.572	0.814	0.916
L3	0.572	0.815	0.916
L4	0.572	0.816	0.916

C Language-Model Support Tables

Table 7: Compact summary of the trainable null result.

Dataset	HOPF / BASE	HOPF - PERM	HOPF eff. paths	PERM eff. paths
PTB	1.081	+0.13	46.0	45.8
WT2	1.081	-0.03	40.9	42.3

Table 8: Auxiliary-loss control. The balance objective pushes the learned router toward near-uniform usage and therefore changes the comparison regime.

Condition	Val. PPL	Eff. paths	Params
BASELINE	154.55	44.0	5.66M
LEARNED_SPARSE	154.78	62.8	5.66M
HOPF	165.59	47.7	5.64M
PERMUTED	165.25	47.1	5.64M

Table 9: Detailed PTB trainable-model results.

Condition	Val. PPL	Eff. paths	$64/\mathrm{eff}_b$	Params
BASELINE	152.26	43.19	1.48×	5.66M
HOPF	164.54	45.96	1.39×	5.64M
PERMUTED	164.41	45.81	1.40×	5.64M

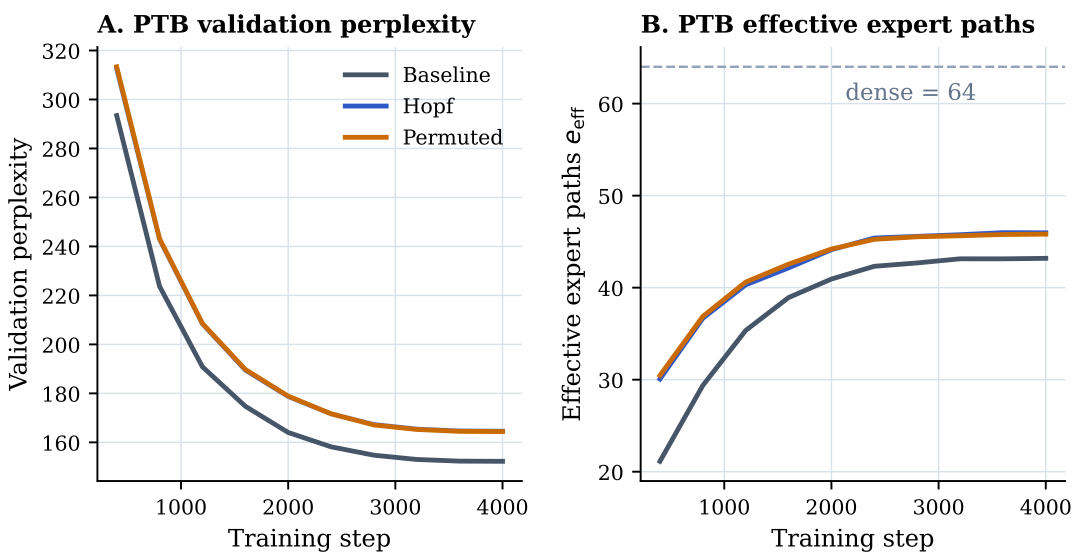

Figure 4: PTB training dynamics. The fixed Hopf and permuted route maps remain close throughout training on both validation perplexity and effective expert-path usage.

Table 10: Detailed WT2 trainable-model results.

Condition	Val. PPL	Std. PPL	Eff. paths	$64/\text{eff}_b$
BASELINE	105.85	1.57	35.37	$1.81\times$
HOPF	114.45	1.14	40.94	$1.56\times$
PERMUTED	114.48	0.87	42.28	$1.51\times$